DSA EXTENDED INTERVIEW PRACTICE BANK

JAVA EDITION • 20 TOPIC GROUPS • EXTENSIVE PROBLEM PRACTICE

495 curated problem entries mapped across your DSA syllabus

HIGH: **443** MEDIUM: **52** LOW: **0**

How to practise every problem

1. First attempt without hints. Write brute force, then optimise.
2. Explain why the algorithm works; state time and space complexity.
3. Code in Java and test empty input, single element, duplicates, boundaries and large constraints.
4. Record mistakes and re-solve after 2 days, 1 week and 3 weeks.
5. For each algorithm, practise the named problems plus at least one variation you create yourself.

Important honesty note

This is an extensive curated practice bank, not a verified archive of every question ever asked. Public interview reports are incomplete and future questions cannot be guaranteed. Company tags and frequency change by role, region and hiring cycle. Difficulty labels are approximate. Classic algorithm exercises without a platform title are marked Core or Advanced.

Priority guide

HIGH = master and solve many variations. MEDIUM = learn and practise thoroughly after core patterns. LOW / Advanced = specialised extensions; prioritise according to target role and available preparation time.

Practice bank

1. Arrays, Prefix/Suffix & Matrix — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Two Sum	Easy	HashMap
Best Time to Buy and Sell Stock	Easy	One-pass minimum
Contains Duplicate	Easy	HashSet
Product of Array Except Self	Medium	Prefix/suffix product
Maximum Subarray	Medium	Kadane
Maximum Product Subarray	Medium	Track min/max DP
Find Minimum in Rotated Sorted Array	Medium	Modified binary search
Rotate Array	Medium	Reversal / cyclic replacement
Merge Sorted Array	Easy	Two pointers from end
Remove Duplicates from Sorted Array	Easy	In-place two pointers
Remove Element	Easy	In-place overwrite
Move Zeroes	Easy	Two pointers
Sort Colors	Medium	Dutch National Flag
Majority Element	Easy	Boyer-Moore
Majority Element II	Medium	Extended Boyer-Moore
Missing Number	Easy	XOR / sum
Find All Numbers Disappeared in an Array	Easy	Index marking
Find the Duplicate Number	Medium	Floyd / binary search
First Missing Positive	Hard	Cyclic placement
Subarray Sum Equals K	Medium	Prefix sum + HashMap
Continuous Subarray Sum	Medium	Prefix remainder
Maximum Size Subarray Sum Equals K	Medium	Prefix sum + earliest index
Maximum Sum Circular Subarray	Medium	Kadane + total sum
Maximum Product of Three Numbers	Easy	Sorting / selection
Longest Consecutive Sequence	Medium	HashSet sequence starts
Insert Delete GetRandom O(1)	Medium	HashMap + array
Set Matrix Zeroes	Medium	In-place row/column markers
Spiral Matrix	Medium	Boundary simulation
Rotate Image	Medium	Transpose + reverse
Diagonal Traverse	Medium	Matrix traversal
Search a 2D Matrix	Medium	Flattened binary search
Search a 2D Matrix II	Medium	Staircase search
Game of Life	Medium	In-place state encoding
Valid Sudoku	Medium	Hash sets / bit masks
Number of Islands II	Hard	DSU on grid
Range Sum Query 2D - Immutable	Medium	2D prefix sum
Maximum Sum of 3 Non-Overlapping Subarrays	Hard	Prefix sums + DP
Candy	Hard	Two-pass greedy
Gas Station	Medium	Greedy reset
Maximum Points You Can Obtain from Cards	Medium	Sliding window complement
Maximum Erasure Value	Medium	Sliding window + set

2. Two Pointers & Sliding Window — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Valid Palindrome	Easy	Two pointers
Valid Palindrome II	Easy	Two pointers with one deletion

Problem	Difficulty	Pattern / algorithm
Two Sum II - Input Array Is Sorted	Medium	Opposite pointers
3Sum	Medium	Sort + two pointers
3Sum Closest	Medium	Sort + two pointers
4Sum	Medium	Sort + nested pointers
Container With Most Water	Medium	Greedy two pointers
Trapping Rain Water	Hard	Two pointers / monotonic stack
Remove Nth Node From End of List	Medium	Fast/slow pointers
Longest Substring Without Repeating Characters	Medium	Variable sliding window
Longest Repeating Character Replacement	Medium	Window + frequency
Permutation in String	Medium	Fixed window + frequency
Find All Anagrams in a String	Medium	Fixed window + frequency
Minimum Window Substring	Hard	Variable window + counts
Sliding Window Maximum	Hard	Monotonic deque
Minimum Size Subarray Sum	Medium	Variable window
Max Consecutive Ones III	Medium	Window with at most K zeros
Fruit Into Baskets	Medium	At most two distinct
Longest Subarray of 1s After Deleting One Element	Medium	Window with one deletion
Subarrays with K Different Integers	Hard	AtMost(K)-AtMost(K-1)
Count Number of Nice Subarrays	Medium	Prefix count / at-most window
Minimum Operations to Reduce X to Zero	Medium	Longest subarray transformation
Number of Sub-arrays of Size K and Average >= Threshold	Easy	Fixed window
Maximize the Confusion of an Exam	Medium	Window with at most K flips

3. Strings & Pattern Matching — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Valid Anagram	Easy	Frequency count
Group Anagrams	Medium	Canonical key
Longest Common Prefix	Easy	Vertical scan / sorting
Longest Palindromic Substring	Medium	Expand around center / DP
Palindromic Substrings	Medium	Expand around center
Longest Palindromic Subsequence	Medium	Interval DP
Valid Parentheses	Easy	Stack
String to Integer (atoi)	Medium	Parsing / overflow
Implement strStr()	Easy	Naive / KMP
Repeated Substring Pattern	Easy	String periodicity
Find the Index of the First Occurrence in a String	Easy	KMP / naive
Zigzag Conversion	Medium	Simulation
Multiply Strings	Medium	Digit multiplication
Add Binary	Easy	Carry simulation
Add Strings	Easy	Two-pointer digit addition
Encode and Decode Strings	Medium	Length-prefix encoding
Decode String	Medium	Stack parsing
Basic Calculator	Hard	Stack / expression parsing
Basic Calculator II	Medium	Operator precedence stack
Compare Version Numbers	Medium	Parsing
Restore IP Addresses	Medium	Backtracking
Word Break	Medium	DP + dictionary
Word Break II	Hard	DFS + memoization
Distinct Subsequences	Hard	2D DP

Problem	Difficulty	Pattern / algorithm
Edit Distance	Medium	2D DP
Regular Expression Matching	Hard	2D DP
Wildcard Matching	Hard	Greedy / DP
KMP Pattern Search	Advanced	Prefix-function / LPS
Z Algorithm Pattern Search	Advanced	Z-function
Rabin-Karp Multiple Pattern Search	Advanced	Rolling hash
Manacher Longest Palindrome	Advanced	Palindrome radii
Longest Duplicate Substring	Hard	Binary search + rolling hash
Palindrome Pairs	Hard	Trie / hashing
Minimum Remove to Make Valid Parentheses	Medium	Stack / two pass
Remove Invalid Parentheses	Hard	BFS / backtracking

4. Hashing & Frequency Patterns — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Two Sum	Easy	Complement lookup
Contains Duplicate II	Easy	HashMap / sliding window
Valid Anagram	Easy	Frequency map
Group Anagrams	Medium	Sorted key / counts
Top K Frequent Elements	Medium	Heap / bucket sort
Top K Frequent Words	Medium	Heap + comparator
Longest Consecutive Sequence	Medium	HashSet
Subarray Sum Equals K	Medium	Prefix sum + map
4Sum II	Medium	Pair-sum frequency
Isomorphic Strings	Easy	Bidirectional mapping
Word Pattern	Easy	Bijective mapping
LRU Cache	Medium	HashMap + doubly linked list
Insert Delete GetRandom O(1)	Medium	HashMap + dynamic array
Design Underground System	Medium	HashMap aggregation
Time Based Key-Value Store	Medium	HashMap + binary search
Find Duplicate Subtrees	Medium	Subtree serialization + map
Copy List with Random Pointer	Medium	Map / interweaving
Ransom Note	Easy	Frequency count
First Unique Character in a String	Easy	Frequency count
Minimum Index Sum of Two Lists	Easy	HashMap
Equal Row and Column Pairs	Medium	Canonical row/column key

5. Binary Search & Ordered Search — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Binary Search	Easy	Classic binary search
Search Insert Position	Easy	Lower bound
First Bad Version	Easy	Boundary search
Find First and Last Position of Element	Medium	Two boundary searches
Search in Rotated Sorted Array	Medium	Modified binary search
Search in Rotated Sorted Array II	Medium	Duplicates handling
Find Minimum in Rotated Sorted Array	Medium	Pivot search
Find Peak Element	Medium	Binary search on slope
Single Element in a Sorted Array	Medium	Parity-based search
Search a 2D Matrix	Medium	Flattened search

Problem	Difficulty	Pattern / algorithm
Koko Eating Bananas	Medium	Binary search on answer
Capacity To Ship Packages Within D Days	Medium	Binary search on answer
Split Array Largest Sum	Hard	Binary search on answer
Aggressive Cows	Medium	Maximize minimum distance
Allocate Books	Medium	Minimize maximum allocation
Median of Two Sorted Arrays	Hard	Binary search partition
Find K-th Smallest Pair Distance	Hard	Binary search + counting
Time Based Key-Value Store	Medium	Binary search over timestamps
Search a Sorted Array of Unknown Size	Medium	Exponential + binary search
Find the Smallest Divisor Given a Threshold	Medium	Binary search on answer
Minimum Number of Days to Make m Bouquets	Medium	Binary search on answer
Magnetic Force Between Two Balls	Medium	Maximize minimum distance

6. Sorting, Selection & Divide and Conquer — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Sort an Array	Medium	Merge sort / quicksort
Merge Sorted Array	Easy	Two pointers
Sort Colors	Medium	Dutch National Flag
Kth Largest Element in an Array	Medium	Quickselect / heap
Top K Frequent Elements	Medium	Bucket / heap
Merge Intervals	Medium	Sort + merge
Largest Number	Medium	Custom comparator
Meeting Rooms	Easy	Sort intervals
Count of Smaller Numbers After Self	Hard	Merge sort counting / BIT
Reverse Pairs	Hard	Merge sort counting
Inversion Count	Medium	Merge sort
Sort List	Medium	Merge sort linked list
Wiggle Sort II	Medium	Sorting / virtual indexing
Maximum Gap	Medium	Bucket sort / radix idea
Relative Sort Array	Easy	Counting / comparator
Minimum Number of Arrows to Burst Balloons	Medium	Sort intervals
The Skyline Problem	Hard	Sweep line + heap
Closest Pair of Points	Advanced	Divide and conquer geometry
Median of Two Sorted Arrays	Hard	Partition / divide and conquer

7. Linked Lists — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Reverse Linked List	Easy	Pointer reversal
Reverse Linked List II	Medium	Sublist reversal
Merge Two Sorted Lists	Easy	Dummy node + merge
Merge k Sorted Lists	Hard	Heap / divide and conquer
Linked List Cycle	Easy	Floyd cycle detection
Linked List Cycle II	Medium	Cycle entry
Middle of the Linked List	Easy	Fast/slow
Remove Nth Node From End of List	Medium	Two pointers
Reorder List	Medium	Middle + reverse + merge
Palindrome Linked List	Easy	Reverse second half
Add Two Numbers	Medium	Carry simulation

Problem	Difficulty	Pattern / algorithm
Add Two Numbers II	Medium	Stacks / reverse
Intersection of Two Linked Lists	Easy	Pointer switching
Copy List with Random Pointer	Medium	HashMap / interweaving
Rotate List	Medium	Circular connection
Partition List	Medium	Two dummy lists
Swap Nodes in Pairs	Medium	Pointer rewiring
Reverse Nodes in k-Group	Hard	Group reversal
Remove Duplicates from Sorted List II	Medium	Dummy node
Flatten a Multilevel Doubly Linked List	Medium	DFS / stack
LRU Cache	Medium	HashMap + doubly linked list
Design Browser History	Medium	Doubly linked list / array

8. Stack, Queue, Deque & Monotonic Stack — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Valid Parentheses	Easy	Stack
Min Stack	Medium	Auxiliary min stack
Implement Stack using Queues	Easy	Queue rotation
Implement Queue using Stacks	Easy	Amortized two stacks
Evaluate Reverse Polish Notation	Medium	Stack evaluation
Daily Temperatures	Medium	Monotonic decreasing stack
Next Greater Element I	Easy	Monotonic stack
Next Greater Element II	Medium	Circular monotonic stack
Online Stock Span	Medium	Monotonic stack
Largest Rectangle in Histogram	Hard	Monotonic increasing stack
Maximal Rectangle	Hard	Histogram per row
Sliding Window Maximum	Hard	Monotonic deque
Asteroid Collision	Medium	Stack simulation
Decode String	Medium	Nested stack parsing
Remove K Digits	Medium	Monotonic stack
132 Pattern	Medium	Monotonic stack
Sum of Subarray Minimums	Medium	Monotonic stack contribution
Sum of Subarray Ranges	Medium	Two monotonic stacks
Trapping Rain Water	Hard	Stack / two pointers
Basic Calculator	Hard	Stack + parsing
Simplify Path	Medium	Stack
Car Fleet	Medium	Sort + stack
Design Circular Queue	Medium	Circular array
Design Front Middle Back Queue	Medium	Two deques

9. Trees & Binary Search Trees — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Maximum Depth of Binary Tree	Easy	DFS / BFS
Minimum Depth of Binary Tree	Easy	BFS / DFS
Same Tree	Easy	Recursive comparison
Symmetric Tree	Easy	Mirror recursion / queue
Invert Binary Tree	Easy	DFS / BFS
Binary Tree Level Order Traversal	Medium	BFS
Binary Tree Zigzag Level Order Traversal	Medium	BFS direction

Problem	Difficulty	Pattern / algorithm
Binary Tree Right Side View	Medium	Level traversal
Binary Tree Vertical Order Traversal	Medium	BFS + ordered map
Binary Tree Inorder Traversal	Easy	Recursive / iterative
Preorder Traversal	Easy	Stack / recursion
Postorder Traversal	Easy	Stack / recursion
Construct from Preorder and Inorder	Medium	Divide and conquer
Construct from Inorder and Postorder	Medium	Divide and conquer
Serialize and Deserialize Binary Tree	Hard	DFS / BFS encoding
Diameter of Binary Tree	Easy	Postorder height DP
Balanced Binary Tree	Easy	Postorder check
Lowest Common Ancestor of Binary Tree	Medium	Postorder recursion
Path Sum	Easy	DFS
Path Sum II	Medium	Backtracking
Binary Tree Maximum Path Sum	Hard	Tree DP
Flatten Binary Tree to Linked List	Medium	Pointer rewiring
Count Good Nodes in Binary Tree	Medium	DFS path maximum
Maximum Width of Binary Tree	Medium	Indexed BFS
All Nodes Distance K in Binary Tree	Medium	Parent map + BFS
Validate Binary Search Tree	Medium	Range constraints
Kth Smallest Element in a BST	Medium	Inorder
Lowest Common Ancestor of a BST	Medium	BST property
Delete Node in a BST	Medium	BST restructuring
Recover Binary Search Tree	Medium	Inorder inversion
Convert Sorted Array to BST	Easy	Divide and conquer
BST Iterator	Medium	Controlled inorder stack
Binary Search Tree to Greater Sum Tree	Medium	Reverse inorder
House Robber III	Medium	Tree DP
Maximum Sum BST in Binary Tree	Hard	Postorder state

10. Heaps & Priority Queues — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Kth Largest Element in an Array	Medium	Heap / quickselect
Kth Largest Element in a Stream	Easy	Min heap
Top K Frequent Elements	Medium	Heap / bucket
K Closest Points to Origin	Medium	Max heap / quickselect
Find Median from Data Stream	Hard	Two heaps
Merge k Sorted Lists	Hard	Min heap
Smallest Range Covering Elements from K Lists	Hard	Heap + pointers
Find K Pairs with Smallest Sums	Medium	Heap frontier
Task Scheduler	Medium	Greedy + max heap
Meeting Rooms II	Medium	Min heap
Reorganize String	Medium	Max heap
Furthest Building You Can Reach	Medium	Min heap
IPO	Hard	Two heaps / greedy
Seat Reservation Manager	Medium	Min heap
Process Tasks Using Servers	Medium	Two priority queues
Single-Threaded CPU	Medium	Sorting + heap

11. Graphs: BFS, DFS, Components & Grid — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Number of Islands	Medium	Grid DFS/BFS
Max Area of Island	Medium	Grid DFS/BFS
Flood Fill	Easy	DFS/BFS
Surrounded Regions	Medium	Boundary DFS/BFS
Clone Graph	Medium	DFS/BFS + map
Rotting Oranges	Medium	Multi-source BFS
Walls and Gates	Medium	Multi-source BFS
01 Matrix	Medium	Multi-source BFS
Pacific Atlantic Water Flow	Medium	Reverse DFS/BFS
Word Ladder	Hard	BFS shortest path
Shortest Path in Binary Matrix	Medium	BFS
Open the Lock	Medium	BFS state search
Number of Provinces	Medium	DFS / DSU
Keys and Rooms	Medium	DFS
Reorder Routes to Make All Paths Lead to the City Zero	Medium	DFS / graph direction
Evaluate Division	Medium	Weighted graph traversal
All Paths From Source to Target	Medium	DFS / backtracking
Path With Maximum Probability	Medium	Dijkstra variant
Find if Path Exists in Graph	Easy	DFS/BFS / DSU
Is Graph Bipartite?	Medium	Two-color BFS/DFS
Possible Bipartition	Medium	Graph coloring
Critical Connections in a Network	Hard	Tarjan bridges

12. Graphs: Topological Sort, Shortest Paths, MST & DSU — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Course Schedule	Medium	Cycle detection / topo sort
Course Schedule II	Medium	Kahn / DFS topo
Alien Dictionary	Hard	Topological ordering
Minimum Height Trees	Medium	Topological leaf trimming
Network Delay Time	Medium	Dijkstra
Path With Minimum Effort	Medium	Dijkstra / binary search
Cheapest Flights Within K Stops	Medium	Bellman-Ford / bounded relaxation
Find the City With the Smallest Number of Neighbors	Medium	Floyd-Warshall
Min Cost to Connect All Points	Medium	Prim / Kruskal
Connecting Cities With Minimum Cost	Medium	MST
Redundant Connection	Medium	DSU
Accounts Merge	Medium	DSU
Number of Operations to Make Network Connected	Medium	DSU
Satisfiability of Equality Equations	Medium	DSU
Graph Valid Tree	Medium	DFS / DSU
Number of Connected Components in an Undirected Graph	Medium	DFS / DSU
Smallest String With Swaps	Medium	DSU
Regions Cut By Slashes	Medium	DSU / graph
Swim in Rising Water	Hard	Dijkstra / binary search
Minimum Cost to Reach Destination in Time	Hard	DP + shortest path
Bellman-Ford implementation	Advanced	Repeated edge relaxation
Floyd-Warshall implementation	Advanced	All-pairs shortest path
0-1 BFS implementation	Advanced	Deque shortest path
Kosaraju SCC implementation	Advanced	Two-pass DFS

Problem	Difficulty	Pattern / algorithm
Tarjan SCC implementation	Advanced	Low-link DFS
Eulerian Path and Circuit	Advanced	Hierholzer
Articulation Points	Advanced	Discovery/low-link
Maximum Flow / Minimum Cut	Advanced	Ford-Fulkerson / Edmonds-Karp
Bipartite Matching	Advanced	Augmenting paths / Hopcroft-Karp

13. Backtracking & Recursion — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Subsets	Medium	Choose-explore-unchoose
Subsets II	Medium	Duplicate pruning
Permutations	Medium	Used array / swap
Permutations II	Medium	Sort + duplicate pruning
Combinations	Medium	Backtracking
Combination Sum	Medium	Reuse candidates
Combination Sum II	Medium	No reuse + duplicate pruning
Combination Sum III	Medium	Bounded choices
Letter Combinations of a Phone Number	Medium	Cartesian product
Generate Parentheses	Medium	Constraint-guided recursion
Palindrome Partitioning	Medium	Backtracking + palindrome
Word Search	Medium	Grid DFS + backtracking
N-Queens	Hard	Columns + diagonals
N-Queens II	Hard	Count solutions
Sudoku Solver	Hard	Constraint backtracking
Restore IP Addresses	Medium	Bounded segmentation
Matchsticks to Square	Medium	Backtracking + pruning
Partition to K Equal Sum Subsets	Medium	Backtracking / bitmask
Expression Add Operators	Hard	Backtracking + parsing
Remove Invalid Parentheses	Hard	BFS / backtracking
Different Ways to Add Parentheses	Medium	Divide and conquer + memo
Beautiful Arrangement	Medium	Backtracking / bitmask
Android Unlock Patterns	Medium	Backtracking + constraints

14. Greedy & Intervals — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Assign Cookies	Easy	Sort + greedy
Lemonade Change	Easy	Greedy simulation
Jump Game	Medium	Reachability greedy
Jump Game II	Medium	Greedy range expansion
Gas Station	Medium	Greedy reset
Hand of Straights	Medium	Ordered frequency map
Merge Intervals	Medium	Sort + merge
Insert Interval	Medium	Interval scan
Non-overlapping Intervals	Medium	Earliest finish
Minimum Number of Arrows to Burst Balloons	Medium	Interval greedy
Meeting Rooms	Easy	Sort intervals
Meeting Rooms II	Medium	Sweep line / heap
Partition Labels	Medium	Last occurrence
Queue Reconstruction by Height	Medium	Sort + insertion

Problem	Difficulty	Pattern / algorithm
Task Scheduler	Medium	Frequency greedy / heap
Candy	Hard	Two-pass greedy
Boats to Save People	Medium	Sort + two pointers
Minimum Number of Refueling Stops	Hard	Max heap greedy
Maximum Units on a Truck	Easy	Sort by units
Course Schedule III	Hard	Greedy + max heap
Job Sequencing with Deadlines	Advanced	Greedy + DSU / heap
Activity Selection	Core	Earliest finish time
Fractional Knapsack	Core	Value/weight ratio
Huffman Coding	Core	Min heap greedy
Minimum Platforms	Core	Sort arrival/departure

15. Dynamic Programming: 1D, Knapsack & Subset — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Climbing Stairs	Easy	1D DP
Min Cost Climbing Stairs	Easy	1D DP
House Robber	Medium	Take/skip DP
House Robber II	Medium	Circular DP
Delete and Earn	Medium	House-robber transformation
Decode Ways	Medium	1D DP
Coin Change	Medium	Unbounded min DP
Coin Change II	Medium	Unbounded count DP
Combination Sum IV	Medium	Order-sensitive DP
Perfect Squares	Medium	DP / number theory
Integer Break	Medium	1D DP
Partition Equal Subset Sum	Medium	0/1 knapsack
Last Stone Weight II	Medium	Subset-sum partition
Target Sum	Medium	Subset-sum transform
Ones and Zeroes	Medium	2D 0/1 knapsack
Profitable Schemes	Hard	Multi-dimensional knapsack
Rod Cutting	Core	Unbounded knapsack
Subset Sum	Core	Boolean knapsack
Count Subsets with Sum K	Core	Counting DP
Minimum Subset Sum Difference	Core	Partition DP
Longest Increasing Subsequence	Medium	DP / patience sorting
Number of Longest Increasing Subsequence	Medium	DP + counts
Maximum Length of Pair Chain	Medium	Greedy / DP
Russian Doll Envelopes	Hard	Sort + LIS

16. Dynamic Programming: Strings, Grid, Interval & State — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Unique Paths	Medium	Grid DP
Unique Paths II	Medium	Grid obstacles
Minimum Path Sum	Medium	Grid DP
Triangle	Medium	Bottom-up DP
Dungeon Game	Hard	Reverse grid DP
Maximal Square	Medium	2D DP
Longest Common Subsequence	Medium	2D sequence DP

Problem	Difficulty	Pattern / algorithm
Edit Distance	Medium	2D string DP
Longest Palindromic Subsequence	Medium	Interval DP
Distinct Subsequences	Hard	Counting DP
Interleaving String	Medium	2D DP
Wildcard Matching	Hard	String DP
Regular Expression Matching	Hard	String DP
Word Break	Medium	Prefix DP
Word Break II	Hard	Memoized DFS
Palindromic Substrings	Medium	Expand / DP
Burst Balloons	Hard	Interval DP
Minimum Cost to Cut a Stick	Hard	Interval DP
Matrix Chain Multiplication	Core	Interval DP
Minimum Score Triangulation of Polygon	Medium	Interval DP
Predict the Winner	Medium	Game / interval DP
Stone Game	Medium	Game DP
Stone Game II	Medium	State DP
Best Time to Buy and Sell Stock with Cooldown	Medium	State-machine DP
Best Time to Buy and Sell Stock III	Hard	State-machine DP
Best Time to Buy and Sell Stock IV	Hard	K-transactions DP
Cherry Pickup II	Hard	3D / two-agent grid DP
Number of Ways to Stay in the Same Place	Hard	State DP
Paint House	Medium	State transition DP
Paint House II	Hard	Optimised state DP
DP on Trees: House Robber III	Medium	Tree DP
Digit DP: count numbers in range	Advanced	Digit position + tight state
Bitmask DP: Travelling Salesperson	Advanced	Subset-state DP

17. Bit Manipulation & Number Theory — HIGH PRIORITY

Problem	Difficulty	Pattern / algorithm
Single Number	Easy	XOR
Single Number II	Medium	Bit counts mod 3
Single Number III	Medium	XOR partition
Number of 1 Bits	Easy	Brian Kernighan
Counting Bits	Easy	DP on bits
Reverse Bits	Easy	Bit construction
Power of Two	Easy	n & (n-1)
Power of Four	Easy	Bit mask
Missing Number	Easy	XOR / sum
Sum of Two Integers	Medium	XOR + carry
Bitwise AND of Numbers Range	Medium	Common prefix
Subsets	Medium	Bitmask enumeration
Gray Code	Medium	Binary-reflected Gray code
Maximum XOR of Two Numbers in an Array	Medium	Binary trie
Maximum XOR With an Element From Array	Hard	Offline queries + trie
Find XOR of All Numbers	Medium	XOR properties
XOR Queries of a Subarray	Medium	Prefix XOR
Pow(x, n)	Medium	Binary exponentiation
Sqrt(x)	Easy	Binary search / Newton method
Happy Number	Easy	Cycle detection

Problem	Difficulty	Pattern / algorithm
Count Primes	Medium	Sieve of Eratosthenes
GCD and LCM implementation	Core	Euclidean algorithm
Prime Factorization	Core	Trial division / sieve SPF
Modular Exponentiation	Core	Fast exponentiation
Modular Inverse	Advanced	Extended Euclid / Fermat
Pascal's Triangle	Easy	Combinatorics
Factorial Trailing Zeroes	Medium	Prime factor counting
Excel Sheet Column Number	Easy	Base conversion

18. Tries & Advanced String Structures — MEDIUM PRIORITY

Problem	Difficulty	Pattern / algorithm
Implement Trie (Prefix Tree)	Medium	Trie insert/search/prefix
Design Add and Search Words Data Structure	Medium	Trie + wildcard DFS
Word Search II	Hard	Trie + backtracking
Replace Words	Medium	Trie prefix replacement
Map Sum Pairs	Medium	Trie + prefix sums
Longest Word in Dictionary	Medium	Trie / sorting
Maximum XOR of Two Numbers in an Array	Medium	Binary trie
Palindrome Pairs	Hard	Trie / hashing
Longest Duplicate Substring	Hard	Suffix / rolling hash
Suffix Array construction	Advanced	Suffix sorting
LCP Array construction	Advanced	Kasai algorithm
Suffix Automaton	Advanced	State construction

19. Advanced Data Structures & Range Queries — MEDIUM PRIORITY

Problem	Difficulty	Pattern / algorithm
Range Sum Query - Immutable	Easy	Prefix sum
Range Sum Query - Mutable	Medium	Fenwick / segment tree
Range Sum Query 2D - Immutable	Medium	2D prefix sum
Count of Smaller Numbers After Self	Hard	Fenwick / merge sort
Reverse Pairs	Hard	Merge sort / BIT
Falling Squares	Hard	Segment tree / coordinate compression
My Calendar I	Medium	Ordered map / segment tree
My Calendar II	Medium	Sweep line / segment tree
Range Module	Hard	Segment tree / ordered intervals
The Skyline Problem	Hard	Sweep line + heap
Online Majority Element In Subarray	Hard	Segment tree / voting
Snapshot Array	Medium	Persistent versioned array
Design Skiplist	Hard	Probabilistic ordered structure
Fenwick Tree implementation	Core	Point update / prefix query
Segment Tree implementation	Core	Build / query / update
Lazy Propagation implementation	Advanced	Range update / query
Sparse Table implementation	Advanced	Static idempotent range query
Disjoint Set Union implementation	Core	Path compression + union by size
Order Statistic Tree	Advanced	Rank / kth element
Treap / B-tree concepts	Advanced	Balanced search structures

20. Mixed Interview Design & Pattern Problems — MEDIUM PRIORITY

Problem	Difficulty	Pattern / algorithm
Design HashMap	Easy	Array / chaining
Design HashSet	Easy	Hashing
Design LRU Cache	Medium	HashMap + doubly linked list
LFU Cache	Hard	HashMap + frequency lists
Min Stack	Medium	Stack design
Time Based Key-Value Store	Medium	HashMap + binary search
RandomizedSet	Medium	Array + HashMap
Median Finder	Hard	Two heaps
Serialize and Deserialize Binary Tree	Hard	Tree encoding
Design Twitter	Medium	Heap + HashMap
Design Hit Counter	Medium	Queue / buckets
Design Tic-Tac-Toe	Medium	Incremental counters
Design Snake Game	Medium	Deque + HashSet
Design Phone Directory	Medium	Queue / set
Insert Delete GetRandom O(1) - Duplicates allowed	Hard	Index sets + array
Find Median from Data Stream	Hard	Two heaps
Logger Rate Limiter	Easy	HashMap + time window
Moving Average from Data Stream	Easy	Queue + running sum
Random Pick with Weight	Medium	Prefix sum + binary search
Reservoir Sampling	Advanced	Streaming random sampling

Algorithm coverage checklist

Arrays / matrices: Two pointers, prefix/suffix, difference arrays, Kadane, cyclic placement, matrix traversal, in-place marking.

Strings: Frequency maps, palindrome expansion, KMP/LPS, Z algorithm, rolling hash, parsing, string DP, Manacher.

Search / sort: Binary search boundaries, answer-space search, merge sort, quicksort, quickselect, counting/bucket/radix concepts.

Linked lists: Dummy node, reversal, fast/slow, Floyd, merge, k-group reversal, random pointer, LRU.

Stack / queue: Expression parsing, monotonic stack, monotonic deque, circular queue, histogram, sliding maximum.

Trees / heaps / tries: DFS/BFS traversals, BST invariants, LCA, tree DP, heaps, two heaps, trie, binary trie.

Graphs: DFS/BFS, multi-source BFS, topo sort, DSU, Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS, MST, SCC, bridges, articulation, Eulerian path, flow.

Backtracking / greedy: Subsets, permutations, pruning, constraint ordering, interval scheduling, exchange reasoning, sweep line, Huffman.

Dynamic programming: 1D/2D, knapsack, subset sum, LIS, LCS, edit distance, grid, interval, tree, state-machine, bitmask, digit DP.

Bit / math: XOR, masks, Brian Kernighan, GCD/LCM, sieve, prime factors, modular arithmetic, fast exponentiation.

Advanced structures: DSU, Fenwick, segment tree, lazy propagation, sparse table, coordinate compression, suffix array, LCP, persistent structures.

Suggested external practice indexes

LeetCode Top Interview 150: https://leetcode.com/studyplan/top-interview-150/
NeetCode Roadmap: https://neetcode.io/roadmap
LeetCode problem set: https://leetcode.com/problemset/

Completion standard: You are interview-ready on a pattern when you can recognise it in an unfamiliar statement, derive the solution, implement it cleanly in Java, test edge cases and justify complexity without looking at notes.